

Работа с файлами, используя язык программирования Python

Разработчик: К. Н. Осипов

2026

Цель работы

Начальное изучение средств работы со списками и строками в языке Питон.

Практическое задание

Самостоятельно проработать ключевые положения настоящего пособия, выполняя указанные в тексте примеры. Сделать краткий конспект.

После проработки ключевых положений выполнить индивидуальное задание. Вариант студента N определяется преподавателем.

Содержание отчета о работе

Отчет оформляется следующим образом:

1. Титульная страница.
2. Отчет о выполнении работы – тексты и результаты работы вводимых примеров.
3. Исходные тексты программ, если таковые создавались в процессе выполнения работы.
4. Выводы.

Отчеты, выполненные не по форме, к рассмотрению не допускаются!

Результаты работы сдаются преподавателю в электронной форме. Название отчета должно состоять из фамилии студента и номера работы (например, Иванов_PZ-03) и содержать внутри:

- отчет (Иванов_Lab02.doc);
- файлы с исходными текстами выполненных программ (fib.py и др.);
- файлы с исходными текстами программы, содержащей решение индивидуального задания (ind.py и др.).

Ключевые положения

1 Общие замечания

1.1 Вопрос именования адресов

В лекции мы рассматривали вопрос работы с именами путей в операционной системе MS Windows. Напомним еще раз, что пути задаются либо с удвоением символа бэкслэш (например `'D:\\txt.txt'`), либо с указанием `r` (*raw*) перед кавычками `r'D:\txt.txt'`. Это делается для того, чтобы избежать случаев, когда символы `\n` или `\t` принимаются за спецсимволы.

1.2 Порядок работы с файлами

Общий порядок работы с файлами выглядит следующим образом – с произвольной переменной с помощью функции `open('путь к файлу', 'режим работы с файлом')` связывается определенный файл, после чего все операции с данным файлом выполняются с помощью этой переменной. В конце операций файл отключается от переменной с помощью функции `имя_переменной.close()`.

Для следующего примера нам понадобится текстовый файл. Создадим его и назовем `1.txt` (используйте любой текстовый редактор) со следующим текстом:

```
1, 2, 3
one two three
this is end of the file
```

Сохраните файл на диск, например в корневую директорию Python `C:\Python25\` – тогда путь к нему будет выглядеть как `C:\Python25\1.txt`.

Для начала посмотрим, как файл подключается к произвольной переменной. В командной строке наберите:

```
>>> a = open('1.txt', 'rb')
```

Этим вы свяжете файл `1.txt` (файл-как-объект) с переменной `a`. Эта переменная является «отражением» файла в пространстве Питона. Проверим это (вывод вашей среды может отличаться):

```
>>> a
<open file '1.txt', mode 'rb' at 0xb780b430>
```

Это указывает на то, что переменная `a` связана с файлом `1.txt`. Причем файл открыт в режиме чтения. Вместо `1.txt` можно указать любой другой путь. Общее правило такое: если указывается полный путь (например `D:\\1.txt` – помним про двойной бэкслэш), Питон будет искать файл по нему; если указывается только имя файла – Питон будет искать его либо в той же папке, в которой находится запущенный скрипт, либо в путях, указанных в его внутренней переменной.

Если мы решим определить тип переменной `a`, это можно будет сделать при помощи функции `type()`:

```
>>> type(a)
<type 'file'>
```

2 Режимы работы с файлом

Режим работы задается с помощью текстовой строки, которая может принимать следующие значения:

- **'r'** – (read) файл открывается для чтения. Файл должен уже существовать.
- **'w'** – (write) файл открывается для записи. Если файл не существует – он будет создан. При этом он будет перезаписан, если там что-то уже находилось.
- **'a'** – (append) если файл не существует – он будет создан. Файл открывается для записи, при этом новая запись будет добавлена к содержимому файла.
- **'r+'** – (read+) позволяет сразу чтение и запись. Если файл не существует – он будет создан. При этом он будет перезаписан, если там что-то уже находилось.
- **'a+'** – (append+) позволяет сразу чтение и запись. Если файл не существует – он будет создан. При этом новая запись будет добавлена к содержимому файла.

Кроме того, к режиму дописывается постфикс, обозначающий, как именно следует читать файл. Если там указано **'b'** (binary), Питон будет работать с файлом как с двоичным файлом любого формата, в противном случае – он будет открыт как текстовый файл. Обратите внимание на то, что мы открыли файл как двоичный (режим **'rb'**) – это основная практика при работе с платформой Windows.

Также обратите внимание на то, что режимы и путь к файлу – это обычные текстовые строки, которые могут быть заданы как явно, так и неявно (в виде переменной, которая формируется в зависимости от результатов вычислений).

3 Методы работы с файлом

Все методы работы с файлом-объектом **a** можно посмотреть с помощью директивы **dir()**:

```
>>> dir(a)
['__class__', '__delattr__', '__doc__', '__enter__', '__exit__',
 '__format__', '__getattribute__', '__hash__', '__init__', '__iter__',
 '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__',
 '__sizeof__', '__str__', '__subclasshook__', 'close', 'closed',
 'encoding', 'errors', 'fileno', 'flush', 'isatty', 'mode', 'name',
 'newlines', 'next', 'read', 'readinto', 'readline', 'readlines', 'seek',
 'softspace', 'tell', 'truncate', 'write', 'writelines', 'xreadlines']
```

С частью из этих функций мы познакомимся ниже.

4 Чтение файла: read()

Функция **read()** считывает файл целиком одной длинной строкой с разделителями **\n** в тех местах, где стоят абзацы. Если указывается количество байтов (например **read(15)**), то считывается именно это количество, если нет – то считывается весь файл целиком.

Прочитаем файл:

```
>>> a.read()
'1, 2, 3\none two three\nthis is end of the file\n'
```

Перемещение «указателя», с которого начинается чтение, вдоль байтовой структуры файла осуществляется с помощью двух функций – `tell()` и `seek()`. Первая из них возвращает число байтов от начала файла до текущего положения, вторая – перемещает указатель на нужное количество байтов. Функция `seek()` задается в форме `seek(смещение, точка_отсчета)`, при этом точка отсчета может принимать следующие значения:

- 0 – начало файла;
- 1 – текущее положение указателя;
- 2 – конец файла.

Давайте попробуем сначала сместить указатель к началу файла:

```
>>> a.seek(0,0)
>>> a.tell()
0L
```

Теперь считаем из файла четыре байта:

```
>>> a.read(4)
'1, 2'
>>> a.tell()
4L
```

Мы считали четыре байта, и указатель сместился на четыре байта к концу файла. Обратите внимание, что в данной кодировке один байт равен одному символу. Это далеко не всегда так (например, это неверно, если файл записан в кодировке Unicode), но в данном случае позволяет легко ориентироваться в работе файловых функций.

```
>>> a.seek(2,1)
>>> a.tell()
6L
>>> s = a.read(7)
>>> s
'3\none t'
```

И так далее.

```
>>> a.seek(-12,2)
>>> a.read()
'of the file\n'
>>> a.seek(14,0)
>>> a.tell()
14L
>>> a.read()
'o three\nthis is end of the file\n'
>>> a.tell()
46L
```

После работы с файлом его нужно закрыть, «отвязав» от переменной `a`:

```
>>> a.close()
>>> a
<closed file '1.txt', mode 'rb' at 0xb780b430>
```

5 Чтение файла: `readlines()`

Функция `read()` является базовой, но не всегда удобной для работы с файлом. Наиболее удобная в Питоне функция называется `readlines()` – она считывает весь файл сразу в виде списка из строк:

```
>>> a = open('1.txt', 'rb')
>>> a.readlines()
['1, 2, 3\n', 'one two three\n', 'this is end of the file\n']
```

Средствами Питона можно, например, создать скрипт, распечатывающий первые буквы каждой строки файла:

```
1 a = open('1.txt', 'rb')
2 s = a.readlines()
3 for i in s:
4     print i[0]
```

Результат работы:

```
1
o
t
```

Обратите внимание на то, что строки записываются в список вместе со знаком перевода строки `\n`. Если вам требуется строка «сама по себе», ее можно обрезать, например с помощью среза:

```
1 a = open('1.txt', 'rb')
2 s = a.readlines()
3
4 print 'all_lines:'
5 print
6 for i in s:
7     print i,
8
9 print 'all_lines_in_one_line:'
10 print
11 for i in s:
12     print i[:-1],
```

Результат работы:

all lines:

```
1, 2, 3
one two three
this is end of the file
all lines in one line:
```

```
1, 2, 3 one two three this is end of the file
```

Существует также функция `readline()`, однако в данном курсе она не рассматривается.

6 Запись в файл: write()

Запись в файл производится с помощью функции `write()`. Вызов функции вида `имя_файла.write(строка)` записывает в файл (дописывает или записывает – определяется режимом открытия файла) строку. Режим открытия файла определяет, как именно будет туда записана строка.

Обратите внимание – при записи в файл нужно самому заботиться о сохранении знаков перевода строки. Также обратите внимание, что чтобы записать в файл что-то «нестрочное», его нужно преобразовать в строку с помощью функции `str()`.

Введем скрипт следующего примера:

```
1 b = open('2.txt', 'wb')
2 for i in range(29):
3     b.write(str(i))
```

Результатом работы будет файл `2.txt` со следующим содержанием:

012345678910111213141516171819202122232425262728

Если мы изменим его:

```
1 b = open('2.txt', 'wb')
2 for i in range(29):
3     b.write(str(i) + ' ')
```

То цифры в файле будут разделены пробелами:

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28

А если мы позаботимся об абзацах:

```
1 b = open('2.txt', 'wb')
2 for i in range(29):
3     b.write(str(i) + '\n')
```

Получим:

0
1
2
3
4
5
...
23
24
25
26
27
28

7 Полный цикл работы чтения-записи файла

Пример ниже иллюстрирует работу с чтением и записью файла. Он меняет местами первую строку файла с последней. В данном случае мы читаем файл `1.txt`, считываем из него информацию, меняем местами первую и последнюю строки файла и записываем получившийся список в файл `3.txt`.

```
1 a = open('1.txt','rb')
2 old = a.readlines()
3 a.close()
4
5 first = old[0]
6 last = old[-1]
7 new = old
8 new[0] = last
9 new[-1] = first
10
11 b = open('3.txt','wb')
12 for i in new:
13     b.write(i)
```

Файл 1.txt (исходный файл):

```
1, 2, 3
one two three
this is end of the file
```

Файл 3.txt (файл с результатом работы программы):

```
this is end of the file
one two three
1, 2, 3
```

Индивидуальное задание

Задайте переменную `spisok`, в которую занесите любые семь элементов из следующего набора данных (выбирается согласно варианту):

1. Список дней недели.
2. Список из семи любых фамилий студентов, присутствующих в аудитории.
3. Список из семи любых названий месяцев.
4. Список из семи любых имен студентов, присутствующих в аудитории.
5. Список из семи любых названий предметов в расписании.
6. Список из семи любых предметов в вашем рюкзаке.
7. Название любых семи улиц города.
8. Название любых семи городов.
9. Название любых семи марок автомобилей.
10. Название любых семи деталей компьютера.
11. Название любых семи фильмов.
12. Название любых семи стран.

- В ручном режиме (из командной строки среды IDLE) создайте файл `D:\myfile.txt` и последовательно запишите в него слова из этого списка. Запротоколируйте свои действия. Проверьте себя, открыв указанный файл в текстовом редакторе.
- Создайте строку `fio`, в которую бы входили через пробел ваши фамилия, имя и отчество – например, "Иванов Иван Иванович".
- Создайте скрипт, который добавит эти данные к ранее созданному файлу `D:\myfile.txt`. Проверьте себя, открыв указанный файл в текстовом редакторе.
- Создайте скрипт, который считывает содержимое файла `D:\myfile.txt` и распечатывает в нем каждую вторую строку.

Вопросы для самопроверки

- Где Питон будет искать файл, если указано только имя файла?
- Каких правил следует придерживаться, задавая путь к файлу в системе MS Windows?
- Опишите последовательность действий с файлом при его открытии. В каком режиме возможно открытие файла? Как указывается путь к нему, какие есть варианты указания такого пути? Приведите пример.
- Перечислите и опишите режимы работы с файлом и их отличия друг от друга.
- Опишите последовательность действий с файлом при его закрытии. Как указывается путь к нему, какие есть варианты указания такого пути? Приведите пример.
- Как в общем случае организуется последовательность чтения-записи из/в файл? Приведите пример.
- Опишите работу команды `read()`. Приведите пример ее использования.
- Опишите работу команды `seek()`. Приведите пример ее использования.
- Опишите работу команды `tell()`. Приведите пример ее использования.
- Опишите работу команды `readline()`. Приведите пример ее использования.
- Опишите работу команды `readlines()`. Приведите пример ее использования.
- Опишите работу команды `write()`. Приведите пример ее использования.
- Какие преимущества предоставляет команда `readlines()` по сравнению с командой `read()`?

Рекомендуемая литература

1 Библиография

1. Г. Россум, Ф.Л.Дж. Дрейк, Д.С. Откидач. Язык программирования Python.
2. Сузи Р.А. Язык программирования Python. Конспект лекций.